

Funding an agent to buy on BotTrunk

Your agent pays for each call with its own USDC on Algorand. Before it can do that, someone has to give it a wallet and put money in it — **an agent cannot fund itself**, and there is no on-ramp inside the loop.

This takes about five minutes. The whole of it is: create a wallet, send it ALGO, send it USDC.

Before you start

You need an Algorand wallet of your own — Defly, Pera, or any exchange that can withdraw on the **Algorand network** — holding:

	Amount	What it is for
ALGO	0.3 (about \$0.10)	Algorand charges rent, not per-call fees. See <i>Why ALGO</i> below.
USDC	whatever you want the agent to spend	Most data services cost \$0.02–\$0.09 per call.

Two things worth checking before you send anything:

- **USDC on Algorand, not on Ethereum or Solana.** The same name, different chains, and a transfer to the wrong one does not arrive. On Algorand, USDC is asset ID **31566704**.
- **MainNet.** BotTrunk serves MainNet only. There is no public TestNet endpoint to rehearse against.

1. Create the agent's wallet

```
npx bottrunk-mcp wallet
```

This generates a brand-new Algorand account and writes it to `~/.bottrunk/wallet.json` with mode 0600. It prints the address and exactly what to send.

That file is the only copy of the key. Treat `~/.bottrunk` like `~/.ssh`: back it up, don't commit it, don't paste it anywhere. BotTrunk never sees it — the key signs a transfer on your machine and only the signed transaction travels to the gateway.

Already run an Algorand account? Skip this whole document: put its 25-word mnemonic in `BOTTRUNK_MNEMONIC` and the agent uses that account directly, with no file, no ALGO to send and no

opt-in to do.

2. Send 0.3 ALGO

Send **0.3 ALGO** to the address it printed. Wait a few seconds for it to land — Algorand blocks are under four seconds.

3. Opt in to USDC

An Algorand account has to opt in to an asset before it can hold it. This happens **by itself** the first time your agent calls a paid tool, so you can usually skip this step.

Do it explicitly if you want to send USDC from a wallet app right away, because Defly and Pera will refuse to compose the transfer until the opt-in exists:

```
npx bottrunk-mcp wallet optin
```

4. Send the USDC

Send the USDC to the same address. Done.

5. Check it

```
npx bottrunk-mcp wallet
```

You want a line like:

```
mainnet 0.299000 ALGO · 5.000000 USDC
```

USDC not opted in means step 3 has not happened yet. USDC 0.000000 **is different and is fine** — it means the account can receive USDC and none has arrived.

One approval instead of three steps

If you would rather do it in a single signature:

```
npx bottrunk-mcp wallet fund --from <your address> --usdc 5
```

This builds one **atomic group** of three transactions — you fund the agent, the agent opts itself in, you deliver the USDC — and checks the whole thing against real chain state with Algorand's `simulate` before anyone is asked to sign. Nothing is submitted by that check and nothing is spent.

All three land or none do, so there is no window in which the wallet has ALGO but cannot yet receive USDC. You sign transactions 0 and 2 with your own wallet; the agent's opt-in is signed already and must be submitted unchanged, or the group id no longer matches. The group expires in about 45 minutes — if you take longer, run it again, it costs nothing.

You need a signer that accepts a transaction group for this. If yours does not, the three steps above work exactly as well.

Why ALGO at all, when the calls are priced in USDC?

Three rules of the chain, none of them BotTrunk's choice:

- 1 **An account must hold at least 0.1 ALGO to exist.** Algorand charges a refundable minimum balance rather than a fee per call.
- 2 **Every asset it holds reserves another 0.1 ALGO.** Holding USDC costs 0.1 ALGO of that minimum balance for as long as you hold it.
- 3 **Only the key holder can sign an opt-in.** Nobody can do it for your agent — not us, not the facilitator, not whoever sends the USDC.

Which is why USDC sent to an account that has not opted in is **rejected, not held**. It does not arrive later; the transfer fails.

We ask for 0.3 rather than the 0.2 those rules require because an account sitting exactly on its minimum balance cannot pay a transaction fee, and some x402 clients pay their own. The spare covers it.

The per-call fee itself is usually free to you: BotTrunk advertises a fee payer, so a client that builds the payment as an atomic group has the facilitator cover the network fee and spends only USDC. A client that signs a plain transfer pays about 0.001 ALGO of its own.

Errors you might see, and what they mean

What you see	What happened
asset missing in destination account (in Defly/Pera)	The agent has not opted in to USDC. Do step 3, then send.
must optin, asset 31566704 missing from ...	The same thing, reported by the chain.

receiver error when sending USDC	Almost always the opt-in again.
balance ... below min	The account is at its minimum balance. Send it a little more ALGO.
This wallet holds 0.000000 ALGO and needs at least 0.3...	The agent refused to sign a payment it cannot make. Do step 2.
No wallet configured	Step 1 has not been run, or BOTTRUNK_HOME points somewhere else.

Spending limits

The agent will not spend more than its caps, checked before anything is signed:

	Default	Set with
Per call	1000 USDC	BOTTRUNK_MAX_PER_CALL
Per day (UTC)	10000 USDC	BOTTRUNK_MAX_PER_DAY

Those defaults are deliberately high, because the catalog includes services that cost hundreds of dollars per call. **If your agent should never spend that much, lower them before you fund the wallet** — they are the only thing standing between an agent and its whole balance.

Fund the agent with what you are willing to lose, the way you would give a contractor a prepaid card rather than your bank login. The wallet is disposable: if you want to retire it, move the USDC out and delete the file.

What happens on a call

The agent asks for a service, gets HTTP 402 with the price, signs a USDC transfer for exactly that amount, and retries. The service runs, the payment settles on Algorand, and the response comes back with the transaction id.

If the request is rejected or the service fails, **nothing is settled and you are not charged** — a 4xx or a 502 never takes money.

Full setup for twelve agent clients: <https://bottrunk.com/connect> Protocol, error contract and limits:
<https://bottrunk.com/docs> Package: <https://www.npmjs.com/package/bottrunk-mcp>